

EMSI — École Marocaine des Sciences de l'Ingénieur

COURS PRATIQUE — POUR DÉBUTANTS

Retrieval-Augmented Generation (RAG)

De zéro à une application RAG fonctionnelle

Module : IA Distribuée & Systèmes Multi-Agents

Niveau : 4ème Année Ingénieur | Semestre 8

Prérequis : Python de base — Aucune connaissance en IA requise

Introduction — À qui s'adresse ce cours ?

Ce cours vous guide pas à pas dans la compréhension et l'implémentation du RAG (Retrieval-Augmented Generation). Il est conçu pour des étudiants sans aucune expérience préalable en intelligence artificielle, embeddings ou bases vectorielles. Chaque concept nouveau est introduit avec une analogie du quotidien avant d'être formalisé.

Ce que vous allez apprendre

- Comprendre POURQUOI le RAG existe (le problème qu'il résout)
- Savoir ce qu'est un embedding (avec une analogie GPS simple)
- Comprendre ce qu'est une base vectorielle (bibliothèque intelligente)
- Maîtriser le chunking (découper des documents en morceaux)
- Construire un chatbot RAG complet avec Python et LangChain
- Évaluer la qualité d'un système RAG (méthode LLM-as-Judge)

Prérequis

- Python de base (savoir lire et écrire des fonctions simples)
- Aucune connaissance en IA, ML, NLP ou mathématiques avancées requise
- Curiosité et envie d'expérimenter !

I. Le Problème — Pourquoi les LLM ont-ils besoin du RAG ?

1.1 Les Grands Modèles de Langage (LLM) : formidables mais limités

Un LLM (Large Language Model) comme ChatGPT, Llama ou Mistral est un programme d'IA entraîné sur des milliards de textes du web, de livres, d'articles. Il peut rédiger, résumer, traduire, coder. C'est comme un étudiant très cultivé qui a lu une bibliothèque entière. Cependant, cet étudiant a un problème majeur...

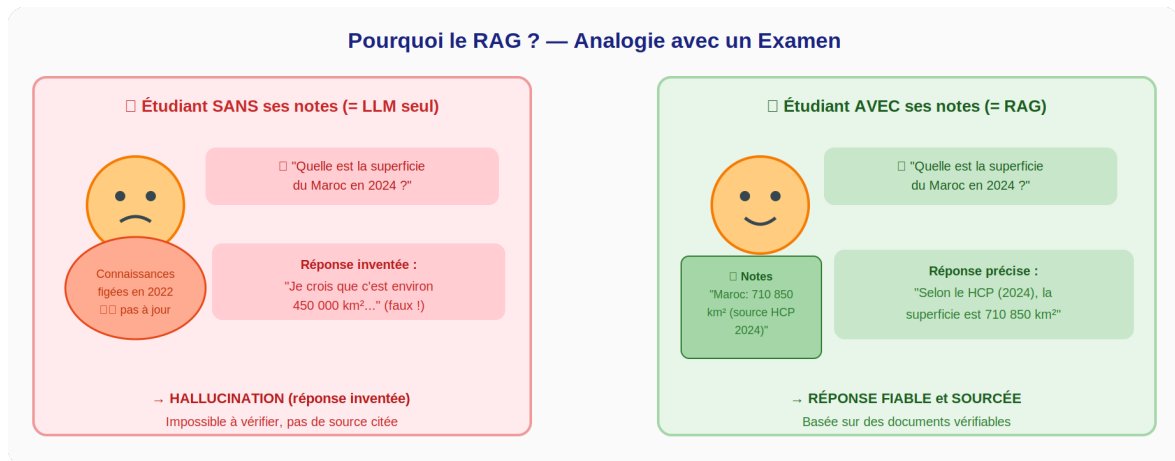


Figure 1 — Analogie de l'examen : un LLM sans RAG "invente" des réponses (hallucination), un LLM avec RAG consulte ses notes (documents) pour répondre avec précision

1.2 Les 3 grandes limitations des LLM

- **Hallucination** : le modèle invente des informations avec une confiance totale. Exemple : si on lui demande le chiffre d'affaires de votre entreprise, il peut donner un chiffre inventé mais plausible.
- **Coupure temporelle (cutoff)** : un LLM est entraîné jusqu'à une date précise. Il ignore tout ce qui s'est passé après. GPT-4 ne connaît pas les événements de 2025.
- **Données privées ignorées** : le LLM n'a jamais vu vos documents internes, vos rapports d'entreprise, vos cours de l'université. Il ne peut pas y répondre.

1.3 La solution : le RAG

Le RAG résout ces trois problèmes en donnant au LLM accès à une base de documents au moment de répondre. Au lieu de tout stocker dans sa "mémoire" (paramètres), il cherche les informations pertinentes à la volée, comme un étudiant qui consulte ses notes pendant l'examen.

❑ Sans RAG	❑ Avec RAG
Invente des réponses (hallucination)	Répond à partir de vos documents
Ne connaît pas vos données privées	Accède à vos PDFs, bases de données
Ne connaît pas l'actualité récente	Documents mis à jour = réponses à jour

Impossible de citer la source	Cite la source exacte
Répond même quand il ne sait pas	Répond "je ne sais pas" si absent du contexte

II. Les 3 Façons d'Exploiter un LLM — Contexte du RAG

Avant d'entrer dans les détails techniques du RAG, il est important de comprendre les trois grandes stratégies pour utiliser un LLM avec des données spécifiques. C'est le cadre présenté dans votre cours par le Pr. Youssfi.

Stratégie	Principe	Quand l'utiliser ?	Avantages / Inconvénients
Few-Shot Learning	On donne des exemples (dataset) directement dans le prompt : {System Message + Exemples + Question}	Peu d'exemples, tâches simples, prototypage rapide	☐ Simple, aucun entraînement ☐ Limité par la taille du prompt
Fine-Tuning	On ré-entraîne le LLM sur votre dataset spécifique. Le LLM devient "spécialisé".	Style d'écriture particulier, connaissances très spécifiques fixes	☐ Performances maximales ☐ Coûteux, données figées
RAG	On laisse le LLM générique mais on lui donne les bons documents au moment de la question.	Données dynamiques, documents privés, actualité	☐ Flexible, sources citables ☐ Données évolutives

Retenir : Pourquoi RAG est souvent le meilleur choix ?

Dans un contexte professionnel ou académique, les données changent souvent.

Exemple : les règlements de l'EMSI, les cours mis à jour chaque année, les données d'entreprise.

Le RAG permet de mettre à jour la base de documents sans ré-entraîner le LLM.

C'est comme changer les livres dans la bibliothèque sans changer le bibliothécaire.

III. Les Embeddings — Transformer les Mots en Nombres

C'est le concept le plus nouveau et le plus important à comprendre. Prenez le temps de lire cette section avec attention.

3.1 Le problème : les ordinateurs ne comprennent pas les mots

Les ordinateurs ne travaillent qu'avec des nombres. Pour qu'un programme puisse "comprendre" un texte, il faut convertir les mots en nombres. La question est : comment représenter le sens d'un texte par des nombres ?

Mauvaise approche : ASCII / code simple

Si on code "A"=1, "B"=2, "C"=3... alors "chien" et "chat" semblent très différents numériquement.

Pourtant, ce sont deux animaux de compagnie proches conceptuellement.

Cette approche ne capture pas le SENS.

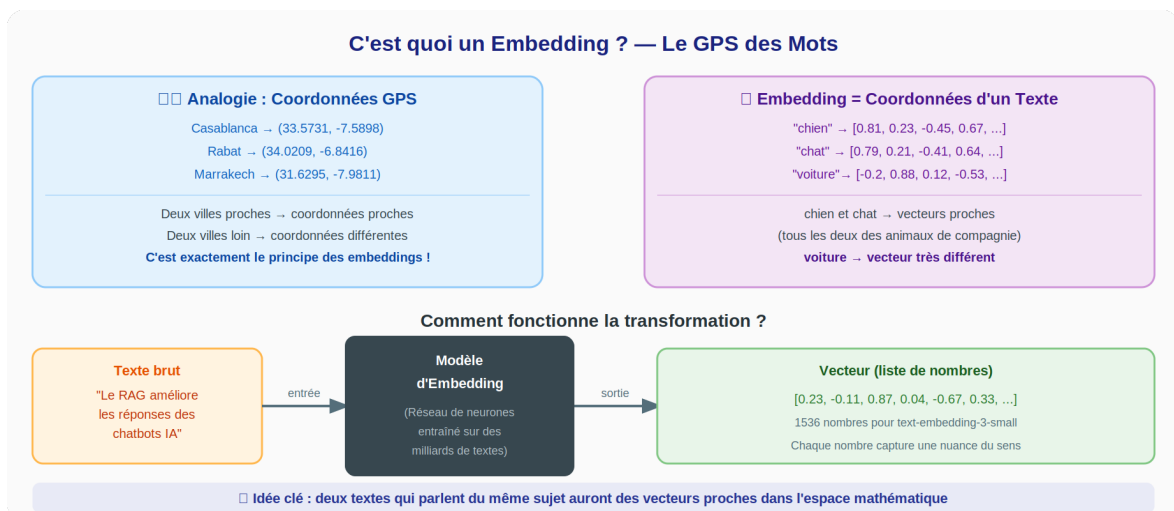


Figure 2 — Analogie GPS : comme les coordonnées géographiques localisent une ville, les embeddings localisent un texte dans un espace mathématique. Textes similaires = coordonnées proches.

3.2 La solution : l'embedding — les "coordonnées GPS" du sens

Un embedding est une liste de nombres (appelée vecteur) qui représente le sens d'un texte. Ce vecteur est calculé par un modèle de réseau de neurones entraîné sur des milliards de textes. La propriété magique :

Propriété fondamentale des embeddings

Deux textes qui parlent du MÊME sujet → vecteurs PROCHES dans l'espace mathématique

Deux textes sans rapport → vecteurs ÉLOIGNÉS

Exemple :

- "Le chien aboie" et "Le caniche fait du bruit" → vecteurs très proches (même sens)

- "Le chien aboie" et "La voiture roule vite" → vecteurs très éloignés (sens différents)

C'est exactement ce qui permet la recherche sémantique (par sens, pas par mot exact) !

3.3 Représentation concrète d'un vecteur

En pratique, un vecteur d'embedding ressemble à ceci. Le modèle text-embedding-ada-002 d'OpenAI produit des vecteurs de 1536 nombres :

```
# Un vecteur d'embedding est simplement une liste de nombres
# Exemple pour le texte "Le RAG améliore les chatbots"

vecteur = [0.23, -0.11, 0.87, 0.04, -0.67, 0.33, 0.51, -0.22, ...]

# ^ ^ ^ ^ ^ ^
# | | | | |
# dimension1 dim2 dim3 dim4 dim5 dim6 ... jusqu'à 1536

# La signification de chaque dimension est apprise automatiquement
# par le modèle pendant son entraînement. On ne les interprète pas
# individuellement. C'est leur COMBINAISON qui capture le sens.
```

3.4 Comment générer un embedding en Python

```
# Option 1 : Utiliser OpenAI (payant mais performant)

from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(model="text-embedding-ada-002")

# Générer l'embedding d'un texte
vecteur = embedding_model.embed_query("Le RAG améliore les chatbots")
print(len(vecteur)) # → 1536 (1536 nombres)
print(vecteur[:5]) # → [0.23, -0.11, 0.87, 0.04, -0.67]

# -----

# Option 2 : Modèle gratuit local (Hugging Face)

from langchain_huggingface import HuggingFaceEmbeddings

# gte-large : modèle open-source recommandé par le cours
```

```
embedding_model = HuggingFaceEmbeddings(
    model_name="thenlper/gte-large"
)

# Même utilisation que ci-dessus
```

3.5 Tableau comparatif des modèles d'embedding

Modèle	Pages/Dollar	Score MTEB	Contexte max	Usage
text-embedding-3-small (OpenAI)	62 500	62.3%	8 192 tokens	Équilibre coût/qualité
text-embedding-3-large (OpenAI)	9 615	64.6%	8 192 tokens	Meilleure précision
text-embedding-ada-002 (OpenAI)	12 500	61.0%	8 192 tokens	Référence classique
gte-large (Hugging Face)	Gratuit	63.1%	512 tokens	Open-source recommandé
multilingual-e5-large	Gratuit	62.7%	512 tokens	Multilingue (fr, ar, en)

Source : MTEB Leaderboard — <https://huggingface.co/spaces/mteb/leaderboard>

IV. La Similarité Cosinus — Comment Comparer Deux Vecteurs ?

Nous savons maintenant que les textes sont convertis en vecteurs. Mais comment savoir si deux vecteurs (donc deux textes) sont "proches" ? C'est là qu'intervient la similarité cosinus.

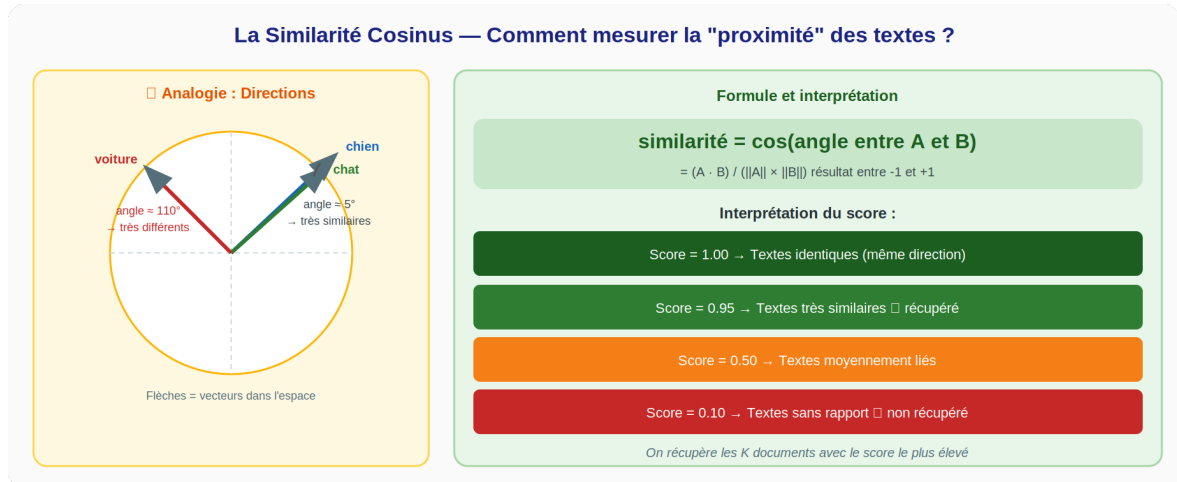


Figure 3 — Analogie des directions : deux vecteurs pointant dans la même direction sont similaires (angle petit, cosinus proche de 1). Deux vecteurs perpendiculaires sont sans rapport (cosinus ≈ 0).

4.1 L'intuition géométrique

Imaginez que chaque texte est une flèche dans un espace à plusieurs dimensions. La similarité cosinus mesure l'angle entre deux flèches :

- Flèches dans la même direction → petit angle → cosinus proche de 1 → textes très similaires
- Flèches perpendiculaires → angle de 90° → cosinus = 0 → textes sans rapport
- Flèches opposées → angle de 180° → cosinus = -1 → textes de sens contraire

4.2 Exemple numérique simple (2 dimensions pour simplifier)

```
# Exemple simplifié en 2 dimensions (au lieu de 1536)
import numpy as np

# Texte A : "Le chien court dans le jardin"
A = np.array([0.8, 0.2])

# Texte B : "Le chien joue dehors" (même sujet)
B = np.array([0.75, 0.25])

# Texte C : "La voiture roule sur l'autoroute" (sujet différent)
C = np.array([-0.1, 0.9])

# Calcul de la similarité cosinus
```

```
def cosine_similarity(a, b):  
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))  
  
print(cosine_similarity(A, B)) # → 0.997 (très similaires □)  
print(cosine_similarity(A, C)) # → 0.12 (très différents □)  
  
# Conclusion : A et B sont récupérés pour une question sur les chiens  
# C ne sera pas récupéré
```

4.3 En pratique : le retriever fait ça automatiquement

Vous n'avez pas besoin de calculer la similarité cosinus manuellement. Le retriever de LangChain le fait automatiquement. Quand vous posez une question :

1. La question est convertie en vecteur (embedding)
2. Ce vecteur est comparé à tous les vecteurs stockés dans la base vectorielle
3. Les K vecteurs les plus proches (score le plus élevé) sont retournés
4. Les textes correspondants sont injectés dans le prompt du LLM

V. Le Chunking — Découper les Documents en Morceaux

Avant de pouvoir calculer des embeddings, il faut découper les documents en morceaux appelés chunks. C'est une étape indispensable car les modèles d'embedding ont une limite de taille (par exemple 512 tokens pour gte-large).

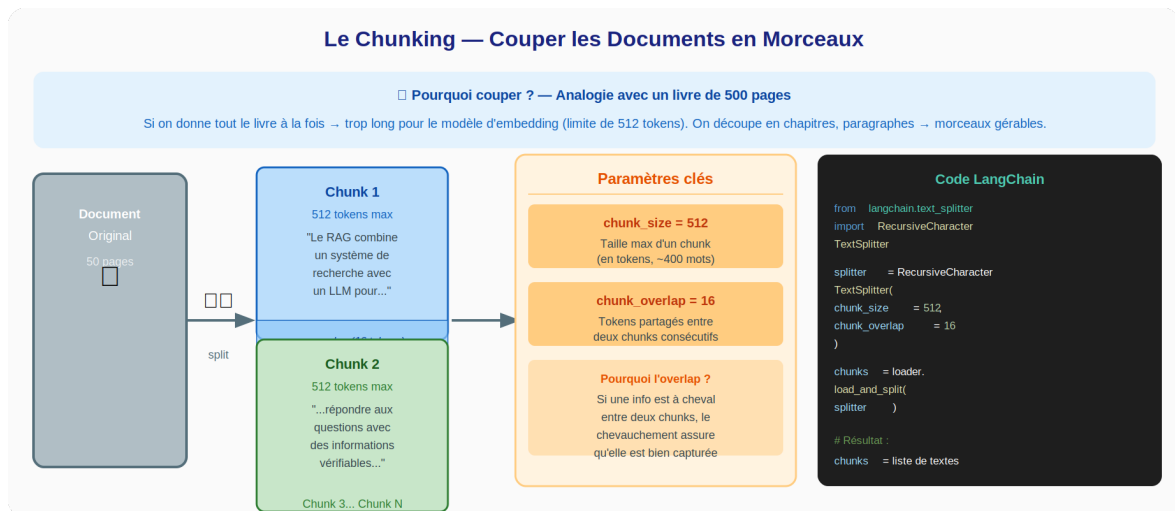


Figure 4 — Le chunking : un document est découpé en morceaux de taille fixe avec un léger chevauchement (overlap) pour ne pas perdre d'information à la frontière entre deux chunks

5.1 Qu'est-ce qu'un token ?

Définition : Token
Un token est la plus petite unité de texte qu'un modèle de langage traite.
Approximativement : 1 token ≈ 0.75 mot en anglais ≈ 0.6 mot en français
Exemples :
• "bonjour" → 1 token
• "intelligence artificielle" → 2 tokens
• "ChatGPT" → 2 tokens (Chat + GPT)
• 512 tokens ≈ 400 mots ≈ 1 page de texte
La limite du modèle gte-large est de 512 tokens par chunk.

5.2 Pourquoi l'overlap (chevauchement) ?

Imaginez qu'une information importante se trouve à cheval entre deux chunks : la fin d'un chunk et le début du chunk suivant. Sans overlap, cette information serait coupée et ni l'un ni l'autre des chunks ne la contiendrait complètement. L'overlap garantit qu'il n'y a pas de perte d'information à la frontière.

Exemple concret d'overlap

Texte : "...les embeddings permettent de représenter le sens des mots. Cette technique est au cœur du RAG..."

SANS overlap (chunk_size=10, overlap=0) :

Chunk 1 : "...les embeddings permettent de représenter le sens des mots."

Chunk 2 : "Cette technique est au cœur du RAG..."

→ Si on cherche "embeddings et RAG", on trouve ni l'un ni l'autre entièrement.

AVEC overlap (chunk_size=10, overlap=3) :

Chunk 1 : "...les embeddings permettent de représenter le sens des mots."

Chunk 2 : "représenter le sens des mots. Cette technique est au cœur du RAG..."

→ Le chunk 2 contient maintenant le lien entre les deux idées ☐

5.3 Code : découpage avec LangChain

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader, PyPDFDirectoryLoader

# — Charger un seul PDF —
loader = PyPDFLoader("./pdfs/mon_document.pdf")

# — Ou charger TOUS les PDFs d'un dossier —
loader = PyPDFDirectoryLoader(path="./pdfs")

# — Configurer le découpage —
text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base", # encodage compatible avec OpenAI
    chunk_size=512, # 512 tokens max par chunk (~400 mots)
    chunk_overlap=16 # 16 tokens partagés entre deux chunks
)

# — Charger ET découper en une seule ligne —
chunks = loader.load_and_split(text_splitter)
```

```
# ——— Résultat ———  
  
print(f"Nombre de chunks : {len(chunks)}") # ex: 347  
print(f"Premier chunk : {chunks[0].page_content[:200]}")  
  
# Chaque chunk est un objet Document avec :  
  
# .page_content → le texte du chunk  
# .metadata → source, numéro de page, etc.
```

VI. La Base Vectorielle — La Bibliothèque Intelligente

Une base vectorielle (Vector Store) est une base de données spécialisée qui stocke des vecteurs et permet de les rechercher efficacement par similarité. C'est le cœur du système RAG.

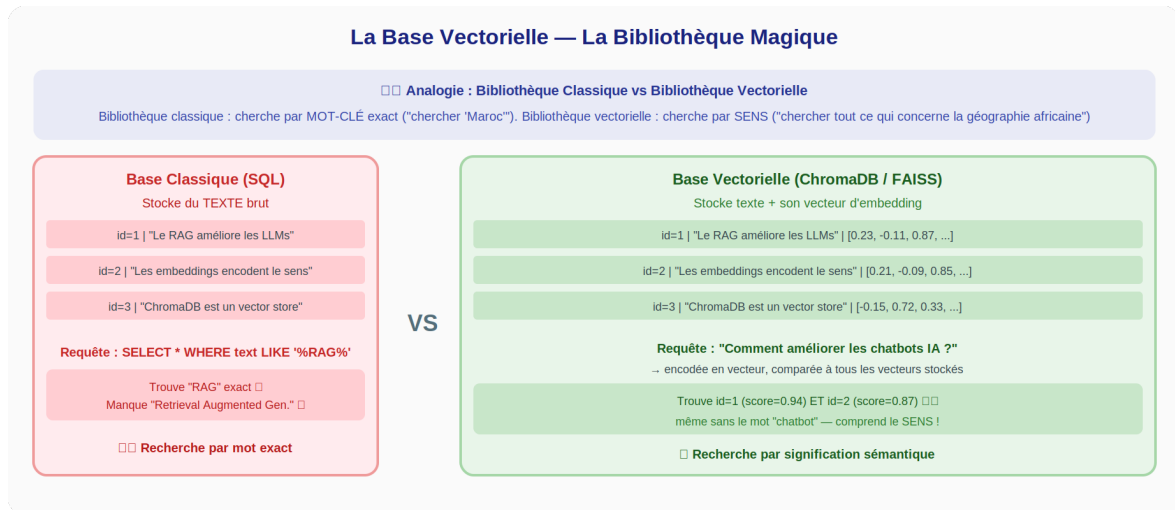


Figure 5 — Base vectorielle vs base classique : la base vectorielle retrouve les documents par SENS (même sans les mots exacts), contrairement à une base SQL classique qui cherche par mots-clés exacts

6.1 Ce que stocke une base vectorielle

Pour chaque chunk de texte, la base vectorielle stocke trois éléments :

1. Le texte original du chunk (ce qu'on retournera au LLM)
2. Le vecteur d'embedding du chunk (ce qui permet la recherche)
3. Les métadonnées : nom du fichier source, numéro de page, date...

6.2 ChromaDB : la base vectorielle du cours

Dans ce cours, nous utilisons ChromaDB, une base vectorielle open-source simple à utiliser, parfaite pour apprendre. Elle fonctionne localement sur votre machine, sans serveur.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

# — 1. Choisir le modèle d'embedding —
embedding_model = OpenAIEmbeddings(model="text-embedding-ada-002")

# — 2. Créer la base vectorielle —

# Chroma indexe automatiquement tous les chunks :

# → calcule l'embedding de chaque chunk

# → stocke texte + vecteur + métadonnées
```

```

vectorstore = Chroma.from_documents(
    chunks, # liste de chunks (étape précédente)
    embedding_model, # modèle pour calculer les vecteurs
    collection_name="Agentic_AI" # nom de la collection
)

# — 3. Créer le retriever —————
# Le retriever est l'interface de recherche
retriever = vectorstore.as_retriever(
    search_type="similarity", # chercher par similarité cosinus
    search_kwargs={"k": 5} # retourner les 5 meilleurs chunks
)

# — 4. Tester la recherche —————
question = "Quelles sont les fonctions d'un agent ?"
docs_pertinents = retriever.invoke(question)

print(f"Nombre de chunks récupérés : {len(docs_pertinents)}") # → 5
for i, doc in enumerate(docs_pertinents):
    print(f"\n--- Chunk {i+1} ---")
    print(f"Source: {doc.metadata['source']}")
    print(f"Page: {doc.metadata['page']}")
    print(f"Contenu: {doc.page_content[:150]}...")

```

6.3 Comparatif des bases vectorielles

Base Vectorielle	Type	Installation	Usage recommandé
ChromaDB	Local, open-source	pip install chromadb	Apprentissage, prototypage
FAISS	Local, open-source	pip install faiss-cpu	R&D, très rapide en local
Pinecone	Cloud SaaS (payant)	Inscription en ligne	Production à grande échelle
Weaviate	Open-source / cloud	Docker ou cloud	Entreprise, hybrid search
pgvector	Extension PostgreSQL	Extension PostgreSQL	Si vous avez déjà PostgreSQL

VII. La Requête et la Génération — Interroger le RAG

Une fois la base vectorielle construite, voyons comment l'interroger. C'est la partie la plus simple : poser une question et obtenir une réponse sourcée.

7.1 Le prompt template — la structure de la question

Un prompt template est un modèle de texte avec des espaces à remplir. En RAG, le template standard contient : les instructions au LLM, le contexte récupéré, et la question de l'utilisateur.

```
# Le prompt template est le "formulaire" qu'on soumet au LLM
prompt_template = """
Réponds à la question suivante EN TE BASANT UNIQUEMENT sur le contexte fourni.
Le contexte est délimité par les balises <context>.
La question est délimitée par les balises <question>.
Si la réponse n'est pas dans le contexte, réponds : "Je ne sais pas."

<context>
{context}
</context>

<question>
{question}
</question>
"""

# {context} et {question} seront remplacés dynamiquement à chaque requête
```

7.2 Étape par étape : une requête RAG

```
from langchain_openai import ChatOpenAI

# — Initialiser le LLM —
llm = ChatOpenAI(model="gpt-4o", temperature=0)
# temperature=0 → réponses déterministes et factuelles (pas créatives)

# — 1. Définir la question —
query = "Quelles sont les fonctions principales d'un agent IA ?"

# — 2. Récupérer les chunks pertinents —
relevant_chunks = retriever.invoke(query)

# → retourne les 5 chunks les plus similaires à la question
```

```
# — 3. Assembler le contexte —
context_list = [chunk.page_content for chunk in relevant_chunks]
context_for_query = ". ".join(context_list)

# → concatène les 5 textes en un seul bloc de contexte

# — 4. Remplir le prompt template —
prompt = prompt_template.format(
    context=context_for_query,
    question=query
)

# — 5. Soumettre au LLM et obtenir la réponse
response = llm.invoke(prompt)
print(response.content)

# → "Un agent IA possède une mémoire, des outils, un LLM..."
```

7.3 La fonction RAG réutilisable

Pour simplifier l'utilisation, on encapsule tout dans une fonction :

```
def RAG(query, llm=llm, prompt_template=prompt_template):
    """Fonction RAG complète : question → réponse sourcée"""

    # 1. Récupérer les chunks pertinents
    context_docs = retriever.invoke(query)

    # 2. Assembler le contexte
    context_list = [doc.page_content for doc in context_docs]
    context_for_query = ". ".join(context_list)

    # 3. Construire et soumettre le prompt
    prompt = prompt_template.format(
        context=context_for_query,
        question=query
    )

    # 4. Obtenir la réponse
    response = llm.invoke(prompt)
    return response.content

# Utilisation :
print(RAG("Quelle est la fonction actuelle du Pr YOUSSEFI ?"))

# → "Professeur à l'ENSET Mohammedia, spécialiste en IA agentique..."
```

```
print(RAG("Qu'est-ce qu'un système multi-agents ?"))  
# → "Un SMA est un système composé d'agents autonomes..."
```

VIII. L'Évaluation — Comment Savoir si le RAG Fonctionne Bien ?

Un bon système RAG ne se juge pas uniquement à l'œil. Il existe des méthodes d'évaluation automatique. La technique LLM-as-Judge utilise un second LLM pour noter la qualité des réponses.

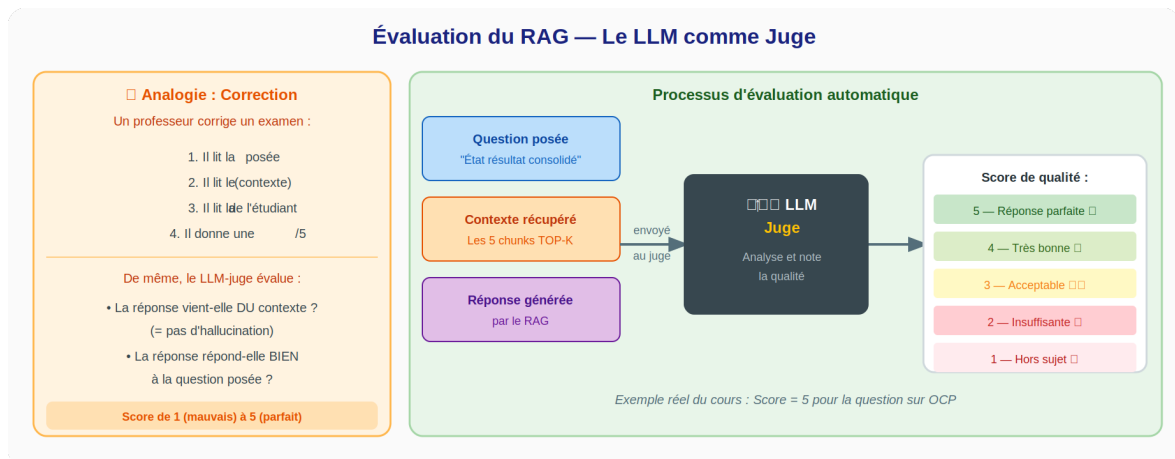


Figure 6 — Le LLM-as-Judge : un deuxième LLM évalue la qualité de la réponse générée, comme un professeur qui corrige. Il vérifie que la réponse vient bien du contexte (pas d'hallucination) et qu'elle répond à la question.

8.1 La métrique Groundedness (ancrage dans les faits)

La métrique principale est le Groundedness : la réponse est-elle entièrement basée sur le contexte fourni ? Un score de 5 signifie que chaque information de la réponse peut être retrouvée dans les chunks récupérés.

```
# — Définir le système d'évaluation —
groundedness_system_message = """
Vous êtes chargé d'évaluer des réponses générées par une IA.
On vous donne : une question, le contexte utilisé, et la réponse de l'IA.

Critères d'évaluation (score de 1 à 5) :
1 — La réponse n'est pas du tout basée sur le contexte
2 — La réponse utilise peu le contexte
3 — La réponse utilise bien le contexte
4 — La réponse est largement fondée sur le contexte
5 — La réponse est entièrement dérivée du contexte

Métrique : La réponse doit être dérivée UNIQUEMENT du contexte fourni.
"""
```

```
# — Template de la question d'évaluation —
user_message_template = """
###Question
{question}
###Context
{context}
###Answer
{answer}
"""

# — Fonction d'évaluation —
def evaluate(system_message, user_message_template, question, model=llm):
    # 1. Récupérer les chunks
    retrieved = retriever.invoke(question)
    context = ". ".join([d.page_content for d in retrieved])
    # 2. Obtenir la réponse du RAG
    answer = RAG(question)
    # 3. Construire le prompt pour le juge
    prompt = f"""{system_message}\nUSER: {user_message_template.format(
    question=question, context=context, answer=answer)}"""
    # 4. Le LLM juge évalue
    juge_response = model.invoke(prompt)
    return juge_response.content

# — Exemple d'utilisation —
question = "État du résultat global consolidé"
score = evaluate(groundedness_system_message, user_message_template, question)
print(score)

# → "Score : 5 – La réponse est entièrement dérivée du contexte ▢"
```

IX. Application Web RAG avec Streamlit

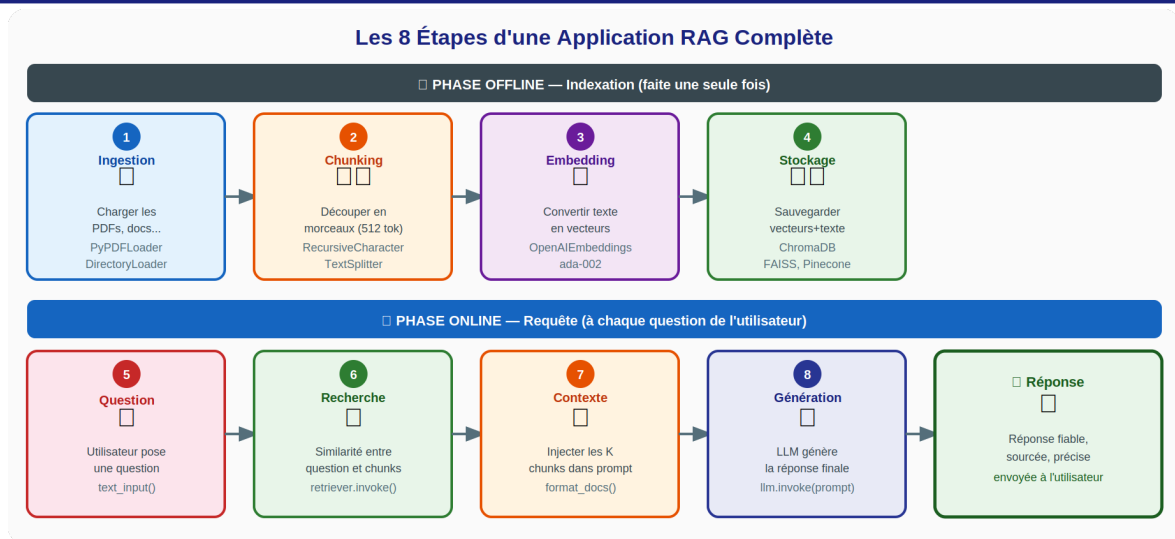


Figure 7 — Les 8 étapes complètes d'une application RAG : phase offline (ingestion, chunking, embedding, stockage) et phase online (question, recherche, contexte, génération)

9.1 Architecture de l'application

Streamlit est une bibliothèque Python qui permet de créer des interfaces web interactives en quelques lignes de code. Dans notre application RAG, Streamlit gère l'interface utilisateur (upload de PDFs, saisie de questions, affichage des réponses).

9.2 Code complet — Application RAG Streamlit

```

"""
Application RAG complète avec Streamlit
Installation : pip install streamlit langchain langchain-openai chromadb pypdf2
Lancement : streamlit run app_rag.py
"""

import streamlit as st
from PyPDF2 import PdfReader
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter
from dotenv import load_dotenv

load_dotenv(override=True)

# — Configuration —
  
```

```
prompt_template = """
Réponds uniquement à partir du contexte fourni.
<context>{context}</context>
<question>{input}</question>
"""

llm = ChatOpenAI(model="gpt-4o", temperature=0)

# — Interface principale —
def main():
    st.set_page_config(page_title="RAG Chatbot", layout="wide")
    st.subheader("▢ Chatbot RAG – Posez vos questions sur vos PDFs",
div
ider="blue")

    # SIDEBAR : Upload des PDFs
    with st.sidebar:
        st.title("▢ Chargement des documents")
        pdf_docs = st.file_uploader(
            label="Uploadez vos PDFs",
            accept_multiple_files=True
        )

        if st.button("▢ Indexer les documents"):
            with st.spinner("Indexation en cours..."):

                # ÉTAPE 1 : Extraire le texte
                content = ""
                for pdf in pdf_docs:
                    reader = PdfReader(pdf)
                    for page in reader.pages:
                        content += page.extract_text()

                # ÉTAPE 2 : Chunking
                splitter = RecursiveCharacterTextSplitter(
                    from_tiktoken_encoder(
                        chunk_size=512, chunk_overlap=16
                    )
                )
                chunks = splitter.split_text(content)
                st.write(f"▢▢ {len(chunks)} chunks créés")
```

```
# ÉTAPE 3 & 4 : Embedding + Vector Store

embedding_model = OpenAIEmbeddings()
vector_store = Chroma.from_texts(
    chunks, embedding_model,
    collection_name="rag_collection"
)

retriever = vector_store.as_retriever(
    kwargs={"k": 5}
)

# Sauvegarder pour la session
st.session_state.retriever = retriever
st.success("▣ Documents indexés !")

# ZONE PRINCIPALE : Chatbot
st.subheader("▣ Chatbot")
user_question = st.text_input("Posez votre question :")

if user_question and "retriever" in st.session_state:
    retriever = st.session_state.retriever
    context_docs = retriever.invoke(user_question)
    context_text = ". ".join([d.page_content for d in context_docs])
    prompt = prompt_template.format(
        context=context_text, input=user_question
    )
    response = llm.invoke(prompt)
    st.write(response.content)
elif user_question:
    st.warning("▣▣ Veuillez d'abord indexer des documents.")

if __name__ == "__main__":
    main()
```

X. RAG Multimodal — Images, Audio et Texte

Le RAG classique traite uniquement du texte. Le RAG Multimodal étend ce principe aux images et aux fichiers audio. C'est une extension naturelle présentée dans votre cours.

10.1 Les trois modalités supportées

Modalité	Pipeline d'ingestion	Modèle utilisé	Exemple d'usage
☐ Texte (PDF)	PDF → Text Splitter → Text Embedding	gte-large, ada-002	Documents, cours, rapports
☐☐ Images	Image → Description sémantique → Text Embedding	CLIP, GPT-4o Vision	Diagrammes, photos, schémas
☐ Audio	Audio → Speech-to-Text → Text Embedding	Whisper (OpenAI)	Réunions, podcasts, cours enregistrés

10.2 Architecture du RAG Multimodal

Dans le RAG multimodal, chaque modalité est convertie en texte (ou en vecteur) avant d'être stockée dans la base vectorielle. Le principe reste le même :

1. Images → un modèle multimodal (CLIP ou GPT-4o) génère une description sémantique → cette description est encodée en vecteur
2. Audio → un modèle de transcription (Whisper) convertit la parole en texte → le texte est encodé en vecteur
3. La recherche est identique : la question de l'utilisateur est comparée à tous les vecteurs (texte, images, audio)

Cas d'usage du RAG Multimodal

- Chatbot sur une base documentaire mixte (PDFs + images + enregistrements)
- Assistant pour analyser des rapports avec graphiques et tableaux
- Système Q&A; sur des cours vidéo enregistrés
- Support technique avec photos de pannes et manuels PDF

XI. Synthèse — Ce que vous avez appris

Concept	En une phrase	Analogie
LLM	Modèle de langage qui génère du texte	Un étudiant très cultivé
Hallucination	Le LLM invente des informations fausses	L'étudiant qui invente à l'examen
RAG	Augmenter le LLM avec des documents	L'étudiant qui consulte ses notes
Embedding	Convertir un texte en liste de nombres	Les coordonnées GPS d'un texte
Similarité cosinus	Mesurer la proximité de deux vecteurs	L'angle entre deux directions
Chunking	Découper un doc en morceaux	Couper un livre en paragraphes
Vector Store	Base de données de vecteurs	Bibliothèque qui cherche par sens
Retriever	Composant de recherche du RAG	Le bibliothécaire qui trouve les livres
Prompt template	Modèle de texte avec variables	Un formulaire à remplir
LLM-as-Judge	LLM qui évalue les réponses	Le professeur qui corrige

Références et Ressources

Pour aller plus loin

- Lewis et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS.
- Documentation LangChain : <https://python.langchain.com>
- MTEB Leaderboard (classement des modèles d'embedding) : <https://huggingface.co/spaces/mteb/leaderboard>
- ChromaDB : <https://www.trychroma.com>
- Streamlit : <https://streamlit.io>
- Modèle gte-large : <https://huggingface.co/thenlper/gte-large>
- OpenAI Embeddings : <https://platform.openai.com/docs/guides/embeddings>

- pgvector (PostgreSQL) : <https://github.com/pgvector/pgvector>